Python Programming Solutions

Short Answers and Program Implementations for Module 4 (Theory & Object-Oriented Programming)

Module 4: Core Theory

1. Explain Python modules.

A module is a file containing Python definitions, functions, and statements ending in the `.py` extension. Modules help break down large programs into smaller, manageable, and reusable code components using the `import` keyword.

```
# Save as helper.py
def greet(name):
    return f"Hello, {name}"

# Main program execution
import helper
print(helper.greet("Alice"))
```

2. Explain random module.

The `random` module provides built-in utilities to perform pseudo-random operations, such as generating numbers, shuffling sequences, or picking unexpected choices.

```
import random
print(random.randint(1, 10))       # Generates random integer between 1 and 10
print(random.choice(['A', 'B', 'C'])) # Picks an item out of a collection array
```

3. Explain time module.

The `time` module provides functions to work with time-related functions, including pausing program execution, fetching epoch timestamps, or measuring computation duration offsets.

```
import time
print(time.time())   # Output: Current Unix epoch timestamp
time.sleep(2)        # Suspends program execution execution for exactly 2 seconds
```

4. Explain math module.

The `math` module gives access to standard underlying mathematical functions and constants defined by the C programming standard library (e.g., trigonometry, logarithms, power calculations).

```
import math
print(math.pi)          # Output: 3.141592653589793
print(math.sqrt(16))    # Output: 4.0
print(math.factorial(5))# Output: 120
```

5. Explain namespaces.

A namespace is a mapping from names (variable identifiers) to objects, implemented internally as Python dictionaries. Namespaces prevent naming collisions by ensuring names are uniquely identifiable within distinct scopes (Local, Enclosing, Global, Built-in).

6. Explain scope and lookup rules.

Scope defines the textual region of a Python program where a namespace is directly accessible. Name lookup follows the strict **LEGB rule**:

- **L (Local):** Names assigned inside the currently running function.
- **E (Enclosing):** Names inside any enclosing non-global functions.
- **G (Global):** Names defined at the top level of the module file.
- **B (Built-in):** Pre-installed names preset inside Python (e.g., `print`, `len`).

7. Explain mutable vs immutable objects.

- **Mutable Objects:** Can change internal value states after instantiation without altering address ids (e.g., `list`, `dict`, `set`).
- **Immutable Objects:** Cannot modify their state in-place. Altering values forces Python to instantiate an entirely new object with a new memory ID (e.g., `int`, `float`, `str`, `tuple`).

8. Explain attributes and dot operator.

Attributes are values or functions associated with an object. The dot operator (`.`) is the syntax used to access these attributes or call methods belonging to a module or class instance.

```
import math
print(math.tau)  # Accessing the constant attribute 'tau' using the dot operator
```

Module 4: Object-Oriented Programming (OOP)

1. Explain classes and objects.

- **Class:** A user-defined blueprint or prototype that specifies the attributes and methods common to all objects of that type.
- **Object:** A distinct, runtime instance of a class that encapsulates actual values and real data.

2. Explain attributes and methods.

- **Attributes:** Variables bound inside a class or object instance that hold specific data properties (e.g., `self.name`).
- **Methods:** Functions defined inside a class body that operate on instances, requiring `self` as their first parameter.

3. Explain instances as arguments.

Python objects can be passed directly as arguments to regular functions or other class methods. The function receives a reference to the original instance, allowing it to read or modify its attributes.

```
class Car:
    def __init__(self, color):
        self.color = color

def paint_red(car_instance):
    car_instance.color = "Red" # Modifies object attribute inside function

my_car = Car("Blue")
paint_red(my_car)
print(my_car.color) # Output: Red
```

4. Explain __str__() method.

The `__str__()` method is a special dunder method used to define a human-readable string representation of an object. It is called automatically when passing an object instance into `print()` or `str()`.

```
class User:
    def __init__(self, username):
        self.username = username
    def __str__(self):
        return f"User Profile: {self.username}"

u = User("coder123")
print(u) # Output: User Profile: coder123
```

5. Explain instances as return values.

Functions or methods can instantiate new objects inside their execution blocks and return those objects to the caller using the `return` statement.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

def create_origin_point():
    return Point(0, 0) # Instantiates and returns an object

origin = create_origin_point()
print(origin.x, origin.y) # Output: 0 0
```

Module 4: Core Program Solutions

1. Class and Object Creation Program

```
class SimpleItem:
    def __init__(self, label):
        self.label = label

    def display(self):
        print(f"Item Label: {self.label}")

# Object instantiation and method execution
item1 = SimpleItem("Gadget")
item1.display()  # Output: Item Label: Gadget
```

2. Student Class Program

```
class Student:
    def __init__(self, name, roll_no, marks_list):
        self.name = name
        self.roll_no = roll_no
        self.marks_list = marks_list

    def calculate_percentage(self):
        if not self.marks_list:
            return 0.0
        return sum(self.marks_list) / len(self.marks_list)

    def __str__(self):
        return f"Student: {self.name} (Roll No: {self.roll_no}), Percentage:
{self.calculate_percentage():.2f}%"

# Verification
s = Student("John Doe", 101, [85, 90, 78, 92])
print(s) # Output: Student: John Doe (Roll No: 101), Percentage: 86.25%
```

3. Bank Account Class Program

```
class BankAccount:
    def __init__(self, account_holder, balance=0.0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited ${amount}. New Balance: ${self.balance}")
        return self.balance

    def withdraw(self, amount):
        if amount > self.balance:
            print("Error: Insufficient funds available.")
        elif amount > 0:
            self.balance -= amount
            print(f"Withdrew ${amount}. Remaining Balance: ${self.balance}")
        return self.balance

# Verification
account = BankAccount("Alice Smith", 500.0)
account.deposit(150.0)
account.withdraw(200.0)
```

4. Distance Class Program

```
class Distance:
    def __init__(self, feet=0, inches=0.0):
        self.feet = feet
        self.inches = inches
        self.normalize()

    def normalize(self):
        # Converts fractional and overflowing inches to clean feet increments
        if self.inches >= 12.0:
            self.feet += int(self.inches // 12)
            self.inches = self.inches % 12.0

    def add_distance(self, other):
        total_feet = self.feet + other.feet
        total_inches = self.inches + other.inches
        return Distance(total_feet, total_inches)

    def __str__(self):
        return f"{self.feet} feet, {self.inches:.1f} inches"

# Verification
d1 = Distance(5, 9.0)
d2 = Distance(3, 5.0)
d3 = d1.add_distance(d2)
print(f"Distance 1: {d1}") # 5 feet, 9.0 inches
print(f"Distance 2: {d2}") # 3 feet, 5.0 inches
print(f"Total Combined Sum: {d3}") # Output: 9 feet, 2.0 inches
```